JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class SplittableRandom

java.lang.Object
    java.util.SplittableRandom

```
public final class SplittableRandom
extends Object
```

A generator of uniform pseudorandom values applicable for use in (among other contexts) isolated parallel computations that may generate subtasks. Class `SplittableRandom` supports methods for producing pseudorandom numbers of type `int`, `long`, and `double` with similar usages as for class `Random` but differs in the following ways:

- Series of generated values pass the DieHarder suite testing independence and uniformity properties of random number generators. (Most recently validated with version 3.31.1.) These tests validate only the methods for certain types and ranges, but similar properties are expected to hold, at least approximately, for others as well. The *period* (length of any series of generated values before it repeats) is at least 2^{64}.
- Method `split()` constructs and returns a new SplittableRandom instance that shares no mutable state with the current instance. However, with very high probability, the values collectively generated by the two objects have the same statistical properties as if the same quantity of values were generated by a single thread using a single `SplittableRandom` object.
- Instances of SplittableRandom are *not* thread-safe. They are designed to be split, not shared, across threads. For example, a `fork/join-style` computation
  using random numbers might include a construction of the form `new Subtask(aSplittableRandom.split()).fork()`.
- This class provides additional methods for generating random streams, that employ the above techniques when used in `stream.parallel()` mode.

Instances of `SplittableRandom` are not cryptographically secure. Consider instead using `SecureRandom` in security-sensitive applications. Additionally, default-

constructed instances do not use a cryptographically random seed unless the system property `java.util.secureRandomSeed` is set to `true`.

Since:

`1.8`

Constructor Summary

Constructors

Constructor	Description
`SplittableRandom()`	Creates a new SplittableRandom instance that is likely to generate sequences of values that are statistically independent of those of any other instances in the current program; and may, and typically does, vary across program invocations.
`SplittableRandom(long seed)`	Creates a new SplittableRandom instance using the specified initial seed.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`DoubleStream`	`doubles()`	Returns an effectively unlimited stream of pseudorandom `double` values from this generator and/or one split from it; each value is between zero (inclusive) and one (exclusive).
`DoubleStream`	`doubles(double randomNumberOrigin, double randomNumberBound)`	Returns an effectively unlimited stream of pseudorandom

		`double` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
`DoubleStream`	**`doubles`**`(long streamSize)`	Returns a stream producing the given `streamSize` number of pseudorandom `double` values from this generator and/or one split from it; each value is between zero (inclusive) and one (exclusive).
`DoubleStream`	**`doubles`**`(long streamSize, double randomNumberOrigin, double randomNumberBound)`	Returns a stream producing the given `streamSize` number of pseudorandom `double` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
`IntStream`	**`ints`**`()`	Returns an effectively unlimited stream of pseudorandom `int` values from this generator

		and/or one split from it.
IntStream	**ints**(int randomNumberOrigin, int randomNumberBound)	Returns an effectively unlimited stream of pseudorandom `int` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
IntStream	**ints**(long streamSize)	Returns a stream producing the given `streamSize` number of pseudorandom `int` values from this generator and/or one split from it.
IntStream	**ints**(long streamSize, int randomNumberOrigin, int randomNumberBound)	Returns a stream producing the given `streamSize` number of pseudorandom `int` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
LongStream	**longs**()	Returns an effectively unlimited stream of pseudorandom `long` values from

		this generator and/or one split from it.
LongStream	**longs**(long streamSize)	Returns a stream producing the given `streamSize` number of pseudorandom `long` values from this generator and/or one split from it.
LongStream	**longs**(long randomNumberOrigin, long randomNumberBound)	Returns an effectively unlimited stream of pseudorandom `long` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
LongStream	**longs**(long streamSize, long randomNumberOrigin, long randomNumberBound)	Returns a stream producing the given `streamSize` number of pseudorandom `long` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
boolean	**nextBoolean**()	Returns a pseudorandom `boolean` value.
double	**nextDouble**()	Returns a pseudorandom

		double value between zero (inclusive) and one (exclusive).
double	**nextDouble**(double bound)	Returns a pseudorandom double value between 0.0 (inclusive) and the specified bound (exclusive).
double	**nextDouble**(double origin, double bound)	Returns a pseudorandom double value between the specified origin (inclusive) and bound (exclusive).
int	**nextInt**()	Returns a pseudorandom int value.
int	**nextInt**(int bound)	Returns a pseudorandom int value between zero (inclusive) and the specified bound (exclusive).
int	**nextInt**(int origin, int bound)	Returns a pseudorandom int value between the specified origin (inclusive) and the specified bound (exclusive).
long	**nextLong**()	Returns a pseudorandom long value.

long	**nextLong**(long bound)	Returns a pseudorandom long value between zero (inclusive) and the specified bound (exclusive).
long	**nextLong**(long origin, long bound)	Returns a pseudorandom long value between the specified origin (inclusive) and the specified bound (exclusive).
SplittableRandom	**split**()	Constructs and returns a new SplittableRandom instance that shares no mutable state with this instance.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

SplittableRandom

```
public SplittableRandom(long seed)
```

Creates a new SplittableRandom instance using the specified initial seed. SplittableRandom instances created with the same seed in the same program generate identical sequences of values.

Parameters:

`seed - the initial seed`

SplittableRandom

```
public SplittableRandom()
```

Creates a new SplittableRandom instance that is likely to generate sequences of values that are statistically independent of those of any other instances in the current program; and may, and typically does, vary across program invocations.

Method Detail

split

```
public SplittableRandom split()
```

Constructs and returns a new SplittableRandom instance that shares no mutable state with this instance. However, with very high probability, the set of values collectively generated by the two objects has the same statistical properties as if the same quantity of values were generated by a single thread using a single SplittableRandom object. Either or both of the two objects may be further split using the `split()` method, and the same expected statistical properties apply to the entire set of generators constructed by such recursive splitting.

Returns:

the new SplittableRandom instance

nextInt

```
public int nextInt()
```

Returns a pseudorandom `int` value.

Returns:

a pseudorandom int value

nextInt

```
public int nextInt(int bound)
```

Returns a pseudorandom `int` value between zero (inclusive) and the specified bound (exclusive).

Parameters:

bound - the upper bound (exclusive). Must be positive.

Returns:

a pseudorandom int value between zero (inclusive) and the bound (exclusive)

Throws:

IllegalArgumentException - if bound is not positive

nextInt

```
public int nextInt(int origin,
                   int bound)
```

Returns a pseudorandom int value between the specified origin (inclusive) and the specified bound (exclusive).

Parameters:

origin - the least value returned

bound - the upper bound (exclusive)

Returns:

a pseudorandom int value between the origin (inclusive) and the bound (exclusive)

Throws:

IllegalArgumentException - if origin is greater than or equal to bound

nextLong

```
public long nextLong()
```

Returns a pseudorandom long value.

Returns:

a pseudorandom long value

nextLong

```
public long nextLong(long bound)
```

Returns a pseudorandom long value between zero (inclusive) and the specified bound (exclusive).

Parameters:

bound - the upper bound (exclusive). Must be positive.

Returns:

a pseudorandom long value between zero (inclusive) and the bound (exclusive)

Throws:

IllegalArgumentException - if bound is not positive

nextLong

```
public long nextLong(long origin,
                     long bound)
```

Returns a pseudorandom long value between the specified origin (inclusive) and the specified bound (exclusive).

Parameters:

origin - the least value returned

bound - the upper bound (exclusive)

Returns:

a pseudorandom long value between the origin (inclusive) and the bound (exclusive)

Throws:

IllegalArgumentException - if origin is greater than or equal to bound

nextDouble

```
public double nextDouble()
```

Returns a pseudorandom double value between zero (inclusive) and one (exclusive).

Returns:

a pseudorandom double value between zero (inclusive) and one (exclusive)

nextDouble

```
public double nextDouble(double bound)
```

Returns a pseudorandom double value between 0.0 (inclusive) and the specified bound (exclusive).

Parameters:

bound - the upper bound (exclusive). Must be positive.

Returns:

a pseudorandom double value between zero (inclusive) and the bound (exclusive)

Throws:

IllegalArgumentException - if bound is not positive

nextDouble

```
public double nextDouble(double origin,
                         double bound)
```

Returns a pseudorandom double value between the specified origin (inclusive) and bound (exclusive).

Parameters:

origin - the least value returned

bound - the upper bound (exclusive)

Returns:

a pseudorandom double value between the origin (inclusive) and the bound (exclusive)

Throws:

IllegalArgumentException - if origin is greater than or equal to bound

nextBoolean

```
public boolean nextBoolean()
```

Returns a pseudorandom boolean value.

Returns:

a pseudorandom boolean value

ints

```
public IntStream ints(long streamSize)
```

Returns a stream producing the given streamSize number of pseudorandom int values from this generator and/or one split from it.

Parameters:

streamSize - the number of values to generate

Returns:

a stream of pseudorandom int values

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero

ints

```
public IntStream ints()
```

Returns an effectively unlimited stream of pseudorandom `int` values from this generator and/or one split from it.

Implementation Note:

This method is implemented to be equivalent to `ints(Long.MAX_VALUE)`.

Returns:

a stream of pseudorandom `int` values

ints

```
public IntStream ints(long streamSize,
                      int randomNumberOrigin,
                      int randomNumberBound)
```

Returns a stream producing the given `streamSize` number of pseudorandom `int` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Parameters:

`streamSize` - the number of values to generate

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `int` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero, or `randomNumberOrigin` is greater than or equal to `randomNumberBound`

ints

```
public IntStream ints(int randomNumberOrigin,
                      int randomNumberBound)
```

Returns an effectively unlimited stream of pseudorandom `int` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Implementation Note:

This method is implemented to be equivalent to `ints(Long.MAX_VALUE, randomNumberOrigin, randomNumberBound)`.

Parameters:

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `int` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `randomNumberOrigin` is greater than or equal to `randomNumberBound`

longs

```
public LongStream longs(long streamSize)
```

Returns a stream producing the given `streamSize` number of pseudorandom `long` values from this generator and/or one split from it.

Parameters:

`streamSize` - the number of values to generate

Returns:

a stream of pseudorandom `long` values

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero

longs

```
public LongStream longs()
```

Returns an effectively unlimited stream of pseudorandom `long` values from this generator and/or one split from it.

Implementation Note:

This method is implemented to be equivalent to `longs(Long.MAX_VALUE)`.

Returns:

a stream of pseudorandom `long` values

longs

```
public LongStream longs(long streamSize,
                        long randomNumberOrigin,
                        long randomNumberBound)
```

Returns a stream producing the given `streamSize` number of pseudorandom `long` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Parameters:

`streamSize` - the number of values to generate

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `long` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero, or `randomNumberOrigin` is greater than or equal to `randomNumberBound`

longs

```
public LongStream longs(long randomNumberOrigin,
                        long randomNumberBound)
```

Returns an effectively unlimited stream of pseudorandom `long` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Implementation Note:

This method is implemented to be equivalent to `longs(Long.MAX_VALUE, randomNumberOrigin, randomNumberBound)`.

Parameters:

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `long` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `randomNumberOrigin` is greater than or equal to `randomNumberBound`

doubles

```
public DoubleStream doubles(long streamSize)
```

Returns a stream producing the given `streamSize` number of pseudorandom `double` values from this generator and/or one split from it; each value is between zero (inclusive) and one (exclusive).

Parameters:

`streamSize - the number of values to generate`

Returns:

`a stream of double values`

Throws:

`IllegalArgumentException - if streamSize is less than zero`

doubles

```
public DoubleStream doubles()
```

Returns an effectively unlimited stream of pseudorandom `double` values from this generator and/or one split from it; each value is between zero (inclusive) and one (exclusive).

Implementation Note:

`This method is implemented to be equivalent to doubles(Long.MAX_VALUE).`

Returns:

`a stream of pseudorandom double values`

doubles

```
public DoubleStream doubles(long streamSize,
                            double randomNumberOrigin,
                            double randomNumberBound)
```

Returns a stream producing the given `streamSize` number of pseudorandom `double` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Parameters:

`streamSize - the number of values to generate`

`randomNumberOrigin - the origin (inclusive) of each random value`

`randomNumberBound - the bound (exclusive) of each random value`

Returns:

a stream of pseudorandom double values, each with the given origin (inclusive) and bound (exclusive)

Throws:

IllegalArgumentException - if streamSize is less than zero

IllegalArgumentException - if randomNumberOrigin is greater than or equal to randomNumberBound

doubles

```
public DoubleStream doubles(double randomNumberOrigin,
                            double randomNumberBound)
```

Returns an effectively unlimited stream of pseudorandom double values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Implementation Note:

This method is implemented to be equivalent to doubles(Long.MAX_VALUE, randomNumberOrigin, randomNumberBound).

Parameters:

randomNumberOrigin - the origin (inclusive) of each random value

randomNumberBound - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom double values, each with the given origin (inclusive) and bound (exclusive)

Throws:

IllegalArgumentException - if randomNumberOrigin is greater than or equal to randomNumberBound

contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.